ALL DATA SCIENCE PROJECTS
(PART - 2)

CONTENTS:

1. Customer Churn Prediction [NON- PIPELINE FORMAT]
2. Customer Segmentation using KMeans
3. E-commerce Sales Prediction [NON-PIPELINE FORMAT]
4. Movie Reviews Sentiment Predictor [NLP Project]

Customer Churn Prediction Project

A Project by Om Satyawan Pathak

Contact: omsatyawanpathakgit@gmail.com

In [1]:

```
#Import the required modules:
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
import warnings
import joblib

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, ExtraTreesClassi
fier
from xgboost import XGBClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score


warnings.filterwarnings("ignore")
```

In [2]:

```
#Import the dataset:
df = pd.read_csv("dataset.csv")
```

In [3]:

```
#Inspect the dataset:

print("Null values:")
print(df.isnull().sum())

print("\n\nNo. of duplicate values:")
print(df.duplicated().sum())

print("\n\nCheck the data-types of all columns:")
print(df.dtypes)

print("\n\nView the basic information of the dataset:")
print(df.info())

print("\n\n\nInspect the first 5 rows of the dataset:")
print(df.head())
```

```
Null values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
```

```
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64


No. of duplicate values:
0


Check the data-types of all columns:
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object


View the basic information of the dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
```

```
memory usage: 1.1+ MB
None



Inspect the first 5 rows of the dataset:
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

                PaymentMethod MonthlyCharges  TotalCharges Churn
0            Electronic check          29.85         29.85    No
1                Mailed check          56.95        1889.5    No
2                Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4            Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
```

In [4]:

```
#Convert the 'TotalCharges' column to numeric-type value:
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")

#Check for imbalance in the target attribute:
df.Churn.value_counts()
```

Out[4]:

```
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

In [5]:

```
df.dtypes
```

Out[5]:

```
customerID           object
gender               object
SeniorCitizen         int64
Partner              object
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
```

```
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges        float64
Churn                object
dtype: object
```

In [6]:

```
#Data Proprocessing:

cols = [
    "OnlineSecurity",
    "OnlineBackup",
    "DeviceProtection",
    "TechSupport",
    "StreamingTV",
    "StreamingMovies",
    "MultipleLines"
]

df[cols] = df[cols].replace("No internet service", "No")
df.drop("customerID", axis=1, inplace=True)

# Encode target: No = 0, Yes = 1
df["Churn"] = df["Churn"].map({"No": 0, "Yes": 1})

print("Shape after cleaning:", df.shape)
display(df.head())


print("Check for imbalance in the target attribute:")
print(df["Churn"].value_counts())
```

```
Shape after cleaning: (7043, 20)
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBac
0	Female	0	Yes	No	1	No	No phone service	DSL	No	
1	Male	0	No	No	34	Yes	No	DSL	Yes	
2	Male	0	No	No	2	Yes	No	DSL	Yes	
3	Male	0	No	No	45	No	No phone service	DSL	Yes	
4	Female	0	No	No	2	Yes	No	Fiber optic	No	

```
Check for imbalance in the target attribute:
Churn
0    5174
1    1869
Name: count, dtype: int64
```

In [7]:

```
#Check for missing values in all columns once again:
df.isnull().sum()
```

Out[7]:

```
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
```

```
tenure                0
PhoneService          0
MultipleLines         0
InternetService       0
OnlineSecurity        0
OnlineBackup          0
DeviceProtection      0
TechSupport           0
StreamingTV           0
StreamingMovies       0
Contract              0
PaperlessBilling      0
PaymentMethod         0
MonthlyCharges        0
TotalCharges         11
Churn                 0
dtype: int64
```

In [8]:

```
df["TotalCharges"].fillna(0,inplace=True)
```

In [9]:

```
#Machine Learning part:
```

In [10]:

```
feature_cols = [
    "gender", "tenure", "Dependents", "SeniorCitizen", "Partner",
    "InternetService", "PhoneService", "TechSupport", "Contract",
    "PaymentMethod", "TotalCharges", "StreamingTV", "StreamingMovies"
]

X = df[feature_cols]
y = df["Churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

num_cols = X_train.select_dtypes(include=["int64", "float64"]).columns.tolist()
cat_cols = X_train.select_dtypes(include=["object"]).columns.tolist()

print("Numerical columns:", num_cols)
print("Categorical columns:", cat_cols)
```

```
Numerical columns: ['tenure', 'SeniorCitizen', 'TotalCharges']
Categorical columns: ['gender', 'Dependents', 'Partner', 'InternetService', 'PhoneService
', 'TechSupport', 'Contract', 'PaymentMethod', 'StreamingTV', 'StreamingMovies']
```

In [11]:

```
#Preprocess the data manually (without using a pipeline):


#Standard-Scale the numeric columns:
scaler = StandardScaler()
X_train_num = pd.DataFrame(
    scaler.fit_transform(X_train[num_cols]),
    columns=num_cols,
    index=X_train.index
)
X_test_num = pd.DataFrame(
    scaler.transform(X_test[num_cols]),
    columns=num_cols,
    index=X_test.index
)
```

```
# One-hot encode categorical columns:
encoder = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
X_train_cat_array = encoder.fit_transform(X_train[cat_cols])
X_test_cat_array = encoder.transform(X_test[cat_cols])

cat_feature_names = encoder.get_feature_names_out(cat_cols)

X_train_cat = pd.DataFrame(
    X_train_cat_array, columns=cat_feature_names, index=X_train.index
)
X_test_cat = pd.DataFrame(
    X_test_cat_array, columns=cat_feature_names, index=X_test.index
)

# Combine scaled numerical and encoded categorical features
X_train_processed = pd.concat([X_train_num, X_train_cat], axis=1)
X_test_processed = pd.concat([X_test_num, X_test_cat], axis=1)

print("Processed training shape:", X_train_processed.shape)
print("Processed testing shape:", X_test_processed.shape)
```

```
Processed training shape: (5634, 27)
Processed testing shape: (1409, 27)
```

In [12]:

```
#Define the classification models:
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "KNN Classifier": KNeighborsClassifier(n_neighbors=8, n_jobs=-1),
    "Extra Trees": ExtraTreesClassifier(
        n_estimators=130, max_depth=9, n_jobs=-1, random_state=42
    ),
    "AdaBoost": AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=6),
        n_estimators=200,
        learning_rate=0.8,
        random_state=42
    ),
    "XGBoost": XGBClassifier(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1,
        tree_method="hist",
        eval_metric="logloss"
    )
}
```

In [13]:

```
#Compare all the ml models using Cross-Validation:
kf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)
scoring = ["accuracy", "f1_weighted"]

results = []

for name, model in models.items():
    print(f"Running {name}...")

    cv_results = cross_validate(
        model,
        X_train_processed,
        y_train,
        cv=kf,
        scoring=scoring,
        n_jobs=-1
    )
```

```
    results.append({
        "Model": name,
        "CV Accuracy": round(np.mean(cv_results["test_accuracy"]), 4),
        "CV F1 Score": round(np.mean(cv_results["test_f1_weighted"]), 4)
    })

results_df = pd.DataFrame(results).sort_values(
    by="CV Accuracy", ascending=False
).reset_index(drop=True)

display(results_df)
```

```
Running Logistic Regression...
Running KNN Classifier...
Running Extra Trees...
Running AdaBoost...
Running XGBoost...
```

	Model	CV Accuracy	CV F1 Score
0	**Logistic Regression**	**0.8017**	**0.7949**
1	**Extra Trees**	**0.7977**	**0.7898**
2	**AdaBoost**	**0.7884**	**0.7819**
3	**KNN Classifier**	**0.7859**	**0.7730**
4	**XGBoost**	**0.7804**	**0.7754**

In [14]:

```
# Select and train the best model using CV Accuracy
best_model_name = results_df.iloc[0]["Model"]
best_model = models[best_model_name]
best_model.fit(X_train_processed, y_train)

print("Best model:", best_model_name)
```

```
Best model: Logistic Regression
```

In [15]:

```
#Make predictions for new data:
single_data = pd.DataFrame([{
    "gender": "Male",
    "Partner": "No",
    "Dependents": "Yes",
    "SeniorCitizen": 0,
    "tenure": 22,
    "PhoneService": "Yes",
    "InternetService": "Fiber optic",
    "TechSupport": "No",
    "Contract": "Month-to-month",
    "PaymentMethod": "Credit card (automatic)",
    "TotalCharges": 1949.4,
    "StreamingTV": "Yes",
    "StreamingMovies": "No"
}])

single_num = pd.DataFrame(
    scaler.transform(single_data[num_cols]),
    columns=num_cols
)
single_cat = pd.DataFrame(
    encoder.transform(single_data[cat_cols]),
    columns=cat_feature_names
)
single_processed = pd.concat([single_num, single_cat], axis=1)

prediction = best_model.predict(single_processed)[0]
```

```
if prediction == 0:
    print("This customer will not churn")
else:
    print("This customer will churn")
```

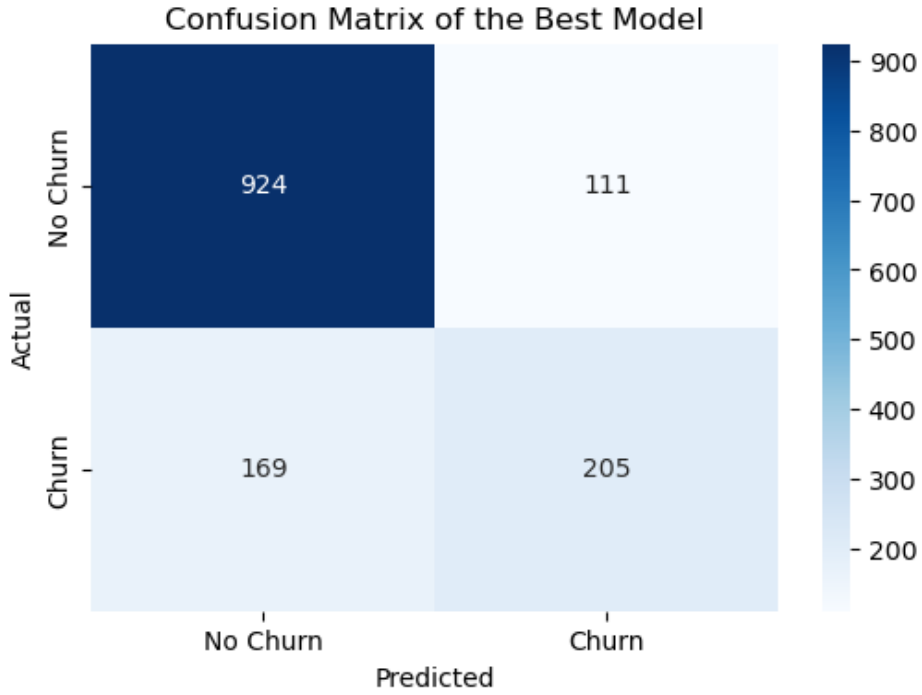
```
This customer will not churn
```

In [16]:

```
#Evaluate the best model on the test set:
y_pred = best_model.predict(X_test_processed)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix of the Best Model")
plt.show()

print("Classification Report of the best model:")
print(classification_report(
    y_test, y_pred, target_names=["No Churn", "Churn"]
))
```




```
Classification Report of the best model:
              precision    recall  f1-score   support

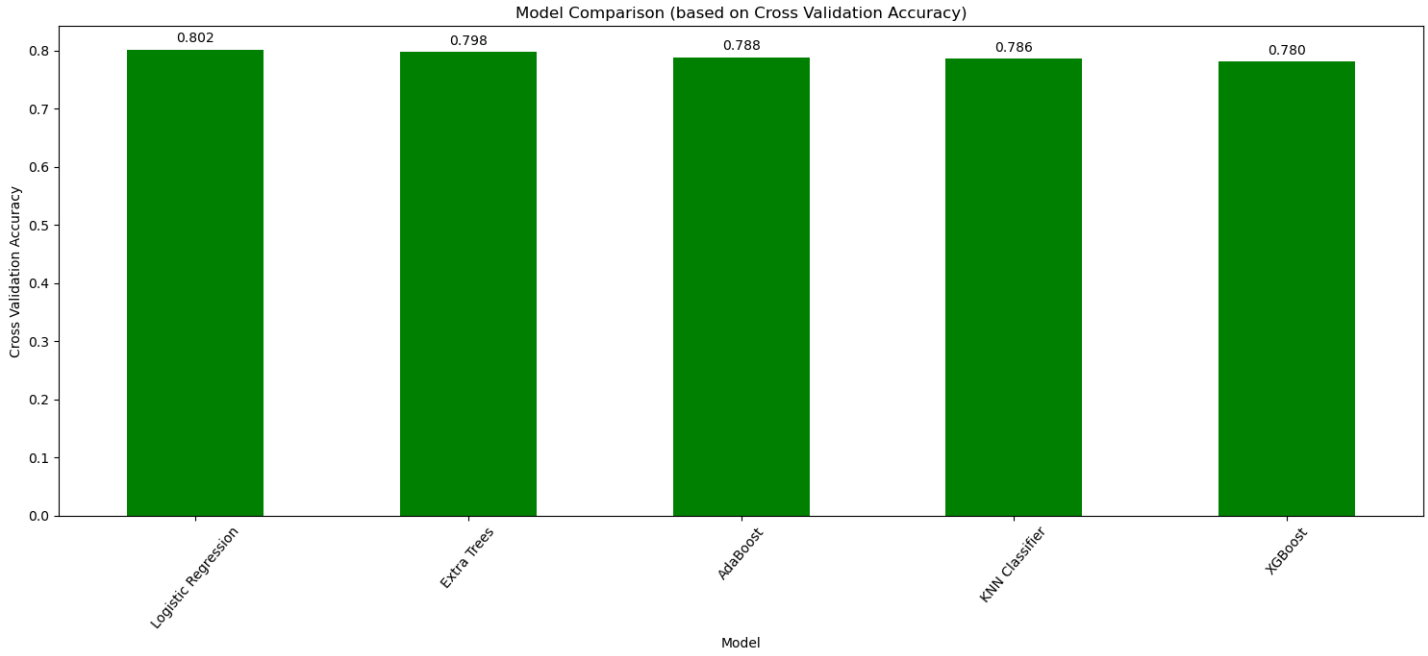
    No Churn       0.85      0.89      0.87      1035
       Churn       0.65      0.55      0.59       374

    accuracy                           0.80      1409
   macro avg       0.75      0.72      0.73      1409
weighted avg       0.79      0.80      0.80      1409
```

In [17]:

```
# Model comparison based on cross-validation accuracy
plt.figure(figsize=(15, 7))
```

```
ax = results_df.set_index("Model")["CV Accuracy"].plot(kind="bar", color="green")
ax.bar_label(ax.containers[0], fmt="%.3f", padding=3)
plt.title("Model Comparison (based on Cross Validation Accuracy)")
plt.xlabel("Model")
plt.ylabel("Cross Validation Accuracy")
plt.xticks(rotation=50)
plt.tight_layout()
plt.show()
```



In [18]:

```
# Save model and preprocessing objects together:
model_bundle = {
    "model": best_model,
    "model_name": best_model_name,
    "scaler": scaler,
    "encoder": encoder,
    "num_cols": num_cols,
    "cat_cols": cat_cols,
    "cat_feature_names": cat_feature_names.tolist(),
    "feature_cols": feature_cols
}

joblib.dump(model_bundle, "CustomerChurnPredictionModel.pkl")
print("Model and preprocessing objects saved successfully.")
```

```
Model and preprocessing objects saved successfully.
```

In [ ]:

```
```

Customer Segmentation Project using KMeans Clustering

- A Project by Om Satyawan Pathak

In [1]:

```
# Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import warnings

warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
```

1. Load the dataset

In [2]:

```
import pandas as pd

df = pd.read_csv("Mall_Customers.csv")

print("\nDataset loaded successfully!")
print("Shape:", df.shape)

df.head()
```

```
Dataset loaded successfully!
Shape: (200, 5)
```

Out[2]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

2. Dataset Information

In [3]:

```
print('Shape:', df.shape)
print('\nColumns:')
print(df.columns.tolist())
print('\nData Types:')
print(df.dtypes)
print('\nMissing Values:')
print(df.isnull().sum())
print('\nDuplicate Rows:', df.duplicated().sum())
```

```
Shape: (200, 5)

Columns:
```

```
['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']

Data Types:
CustomerID                  int64
Gender                     object
Age                         int64
Annual Income (k$)          int64
Spending Score (1-100)      int64
dtype: object

Missing Values:
CustomerID                 0
Gender                     0
Age                        0
Annual Income (k$)         0
Spending Score (1-100)     0
dtype: int64

Duplicate Rows: 0
```

In [4]:

```
df.describe()
```

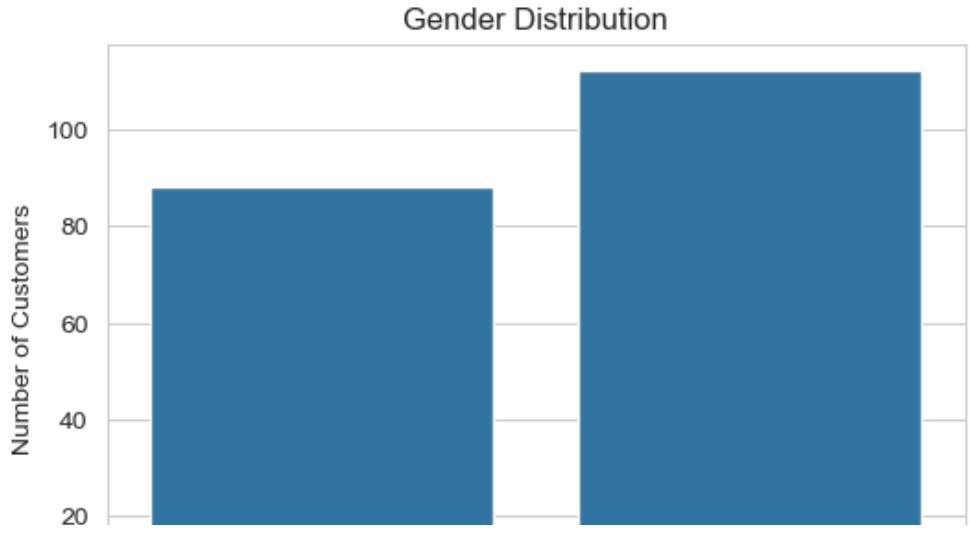
Out[4]:

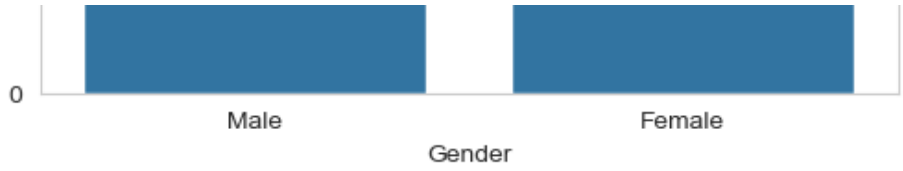
	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

3. Exploratory Data Analysis

In [5]:

```
plt.figure(figsize=(6,4))
sns.countplot(data=df, x='Gender')
plt.title('Gender Distribution')
plt.xlabel('Gender')
plt.ylabel('Number of Customers')
plt.show()
```





In [6]:

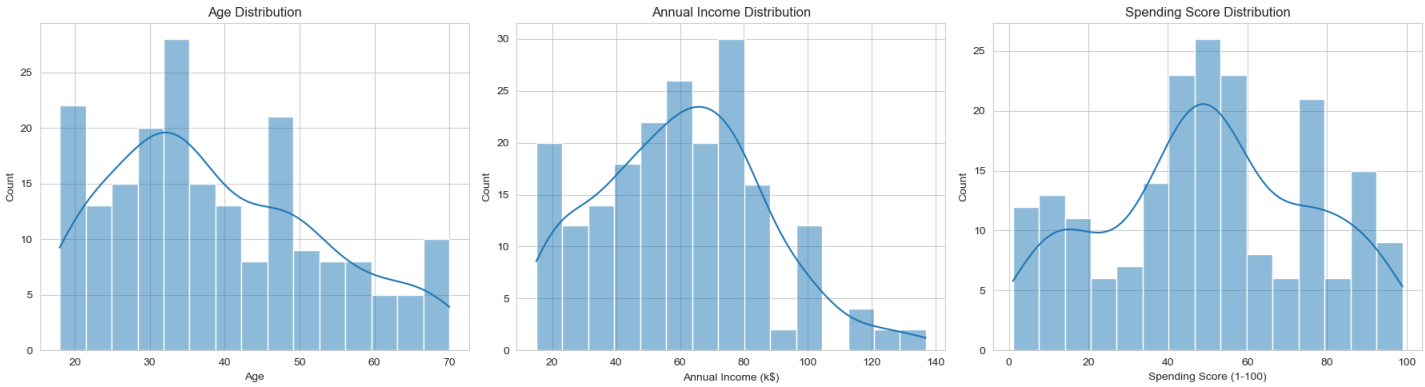
```
fig, axes = plt.subplots(1, 3, figsize=(18,5))

sns.histplot(df['Age'], bins=15, kde=True, ax=axes[0])
axes[0].set_title('Age Distribution')

sns.histplot(df['Annual Income (k$)'], bins=15, kde=True, ax=axes[1])
axes[1].set_title('Annual Income Distribution')

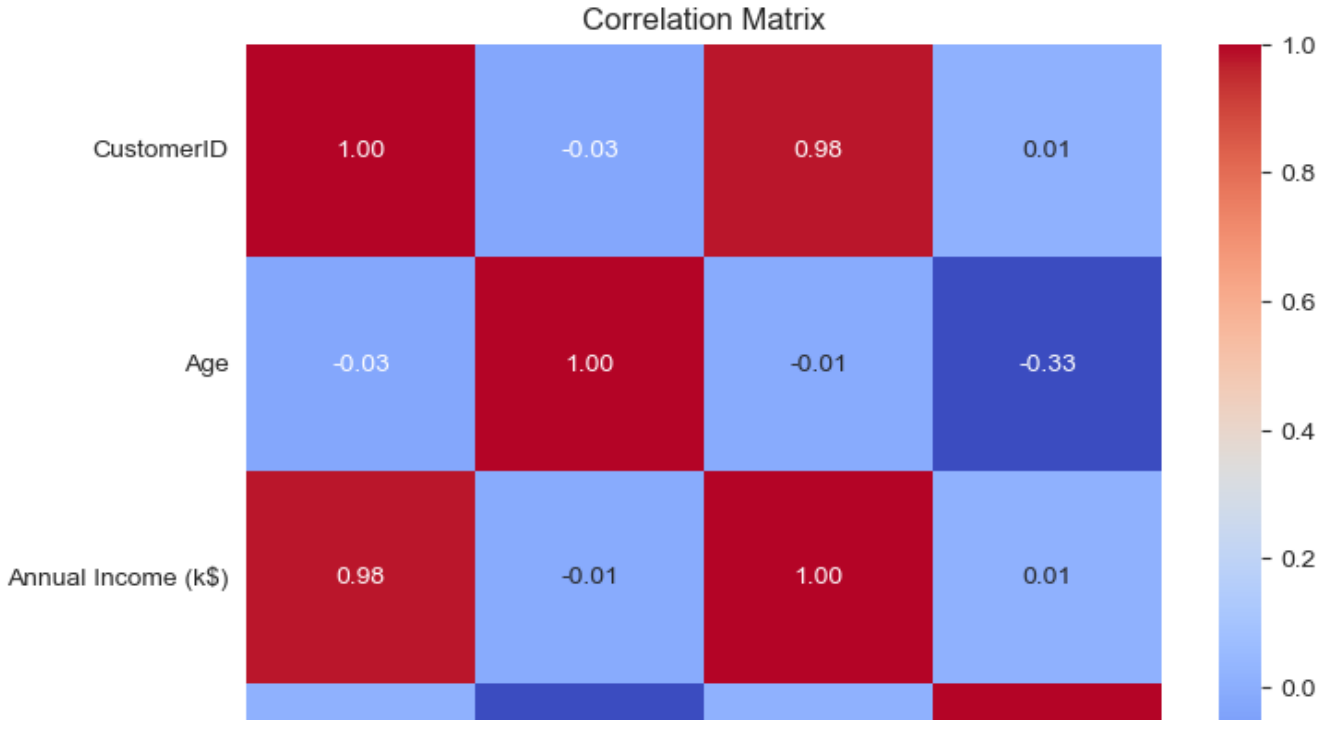
sns.histplot(df['Spending Score (1-100)'], bins=15, kde=True, ax=axes[2])
axes[2].set_title('Spending Score Distribution')

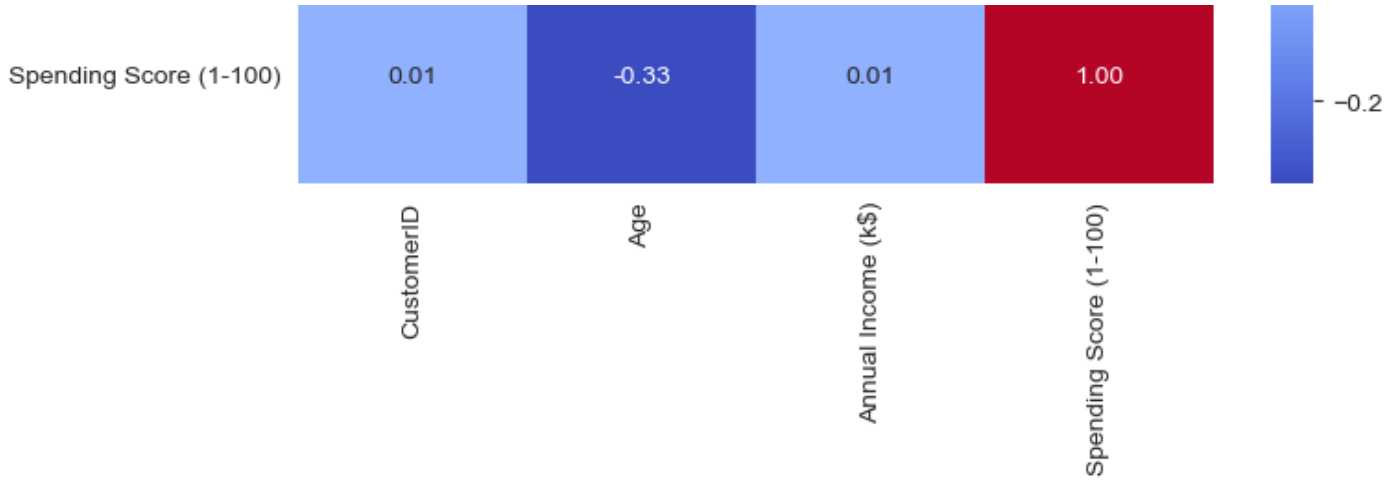
plt.tight_layout()
plt.show()
```



In [7]:

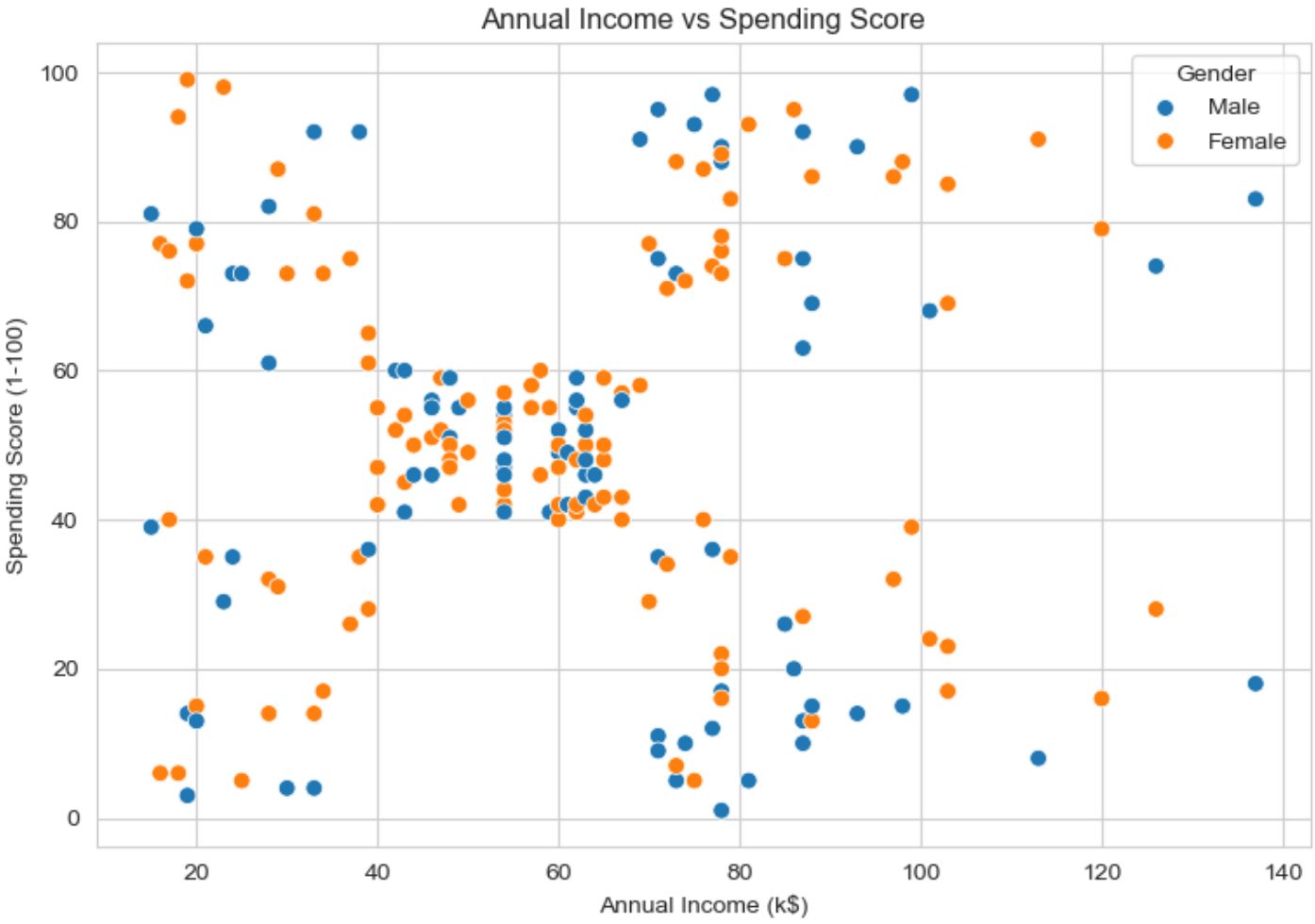
```
plt.figure(figsize=(8,6))
sns.heatmap(
    df.select_dtypes(include=np.number).corr(),
    annot=True,
    fmt='.2f',
    cmap='coolwarm'
)
plt.title('Correlation Matrix')
plt.show()
```





In [8]:

```
plt.figure(figsize=(9,6))
sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Gender',
    s=50
)
plt.title('Annual Income vs Spending Score')
plt.show()
```



4. Feature Selection

We use Annual Income and Spending Score for customer segmentation.

In [9]:

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print('Feature matrix shape:', X.shape)
```

```
Feature matrix shape: (200, 2)
```

5. Elbow Method and Silhoutte Score:

In [10]:

```
wcss = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        init='k-means++',
        random_state=42,
        n_init=10
    )
    labels = kmeans.fit_predict(X_scaled)

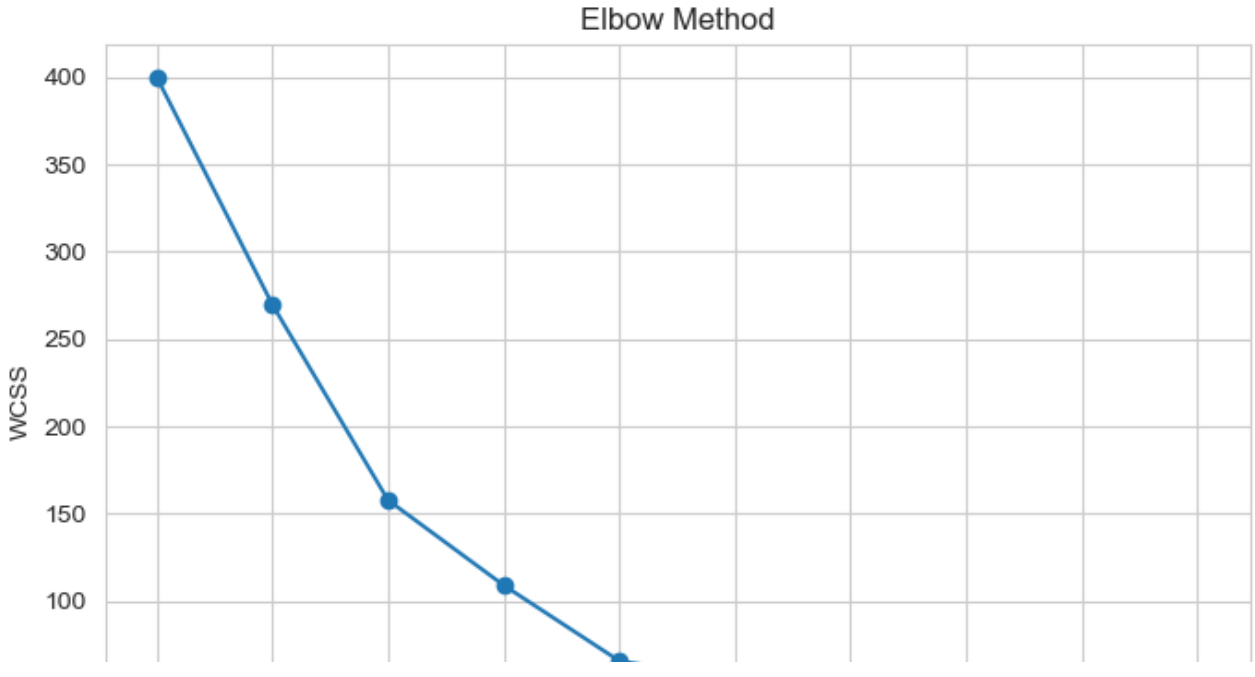
    wcss.append(kmeans.inertia_)

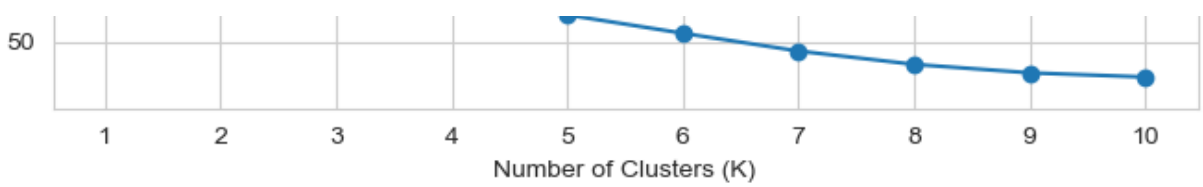
    if k >= 2:
        score = silhouette_score(X_scaled, labels)
        print(f"For K = {k} ----> Silhouette Score = {score:.5f}")



plt.figure(figsize=(8,5))
plt.plot(range(1,11), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.xticks(range(1,11))
plt.show()
```

```
For K = 2 ----> Silhouette Score = 0.32127
For K = 3 ----> Silhouette Score = 0.46658
For K = 4 ----> Silhouette Score = 0.49391
For K = 5 ----> Silhouette Score = 0.55466
For K = 6 ----> Silhouette Score = 0.53988
For K = 7 ----> Silhouette Score = 0.52815
For K = 8 ----> Silhouette Score = 0.45521
For K = 9 ----> Silhouette Score = 0.45709
For K = 10 ----> Silhouette Score = 0.44317
```





6. K-Means Clustering with K = 5

In [11]:

```
kmeans = KMeans(
    n_clusters=5,
    init='k-means++',
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)
df.head(10)
```

Out[11]:

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4
5	6	Female	22	17	76	2
6	7	Female	35	18	6	4
7	8	Female	23	18	94	2
8	9	Male	64	19	3	4
9	10	Female	30	19	72	2

In [12]:

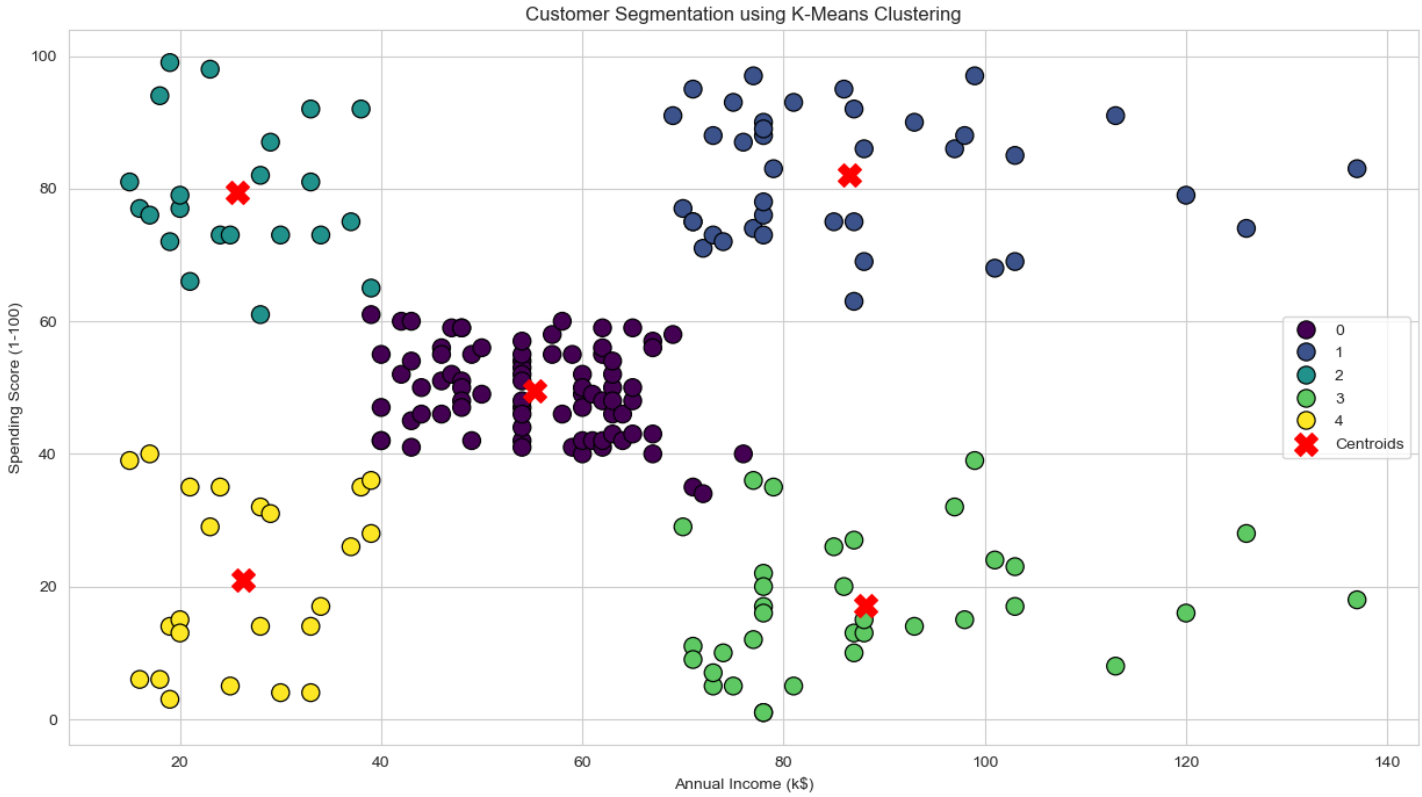
```
plt.figure(figsize=(15,8))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='viridis',
    edgecolors='black',
    s=120
)

centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
    centers[:, 0],
    centers[:, 1],
    s=200,
    marker='X',
    c='red',
    label='Centroids'
)

plt.title('Customer Segmentation using K-Means Clustering')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
```

```
plt.show()
```



7. Cluster Summary

In [13]:

```
cluster_summary = df.groupby('Cluster').agg(
    Customers=('CustomerID', 'count'),
    Average_Age=('Age', 'mean'),
    Average_Income=('Annual Income (k$)', 'mean'),
    Average_Spending=('Spending Score (1-100)', 'mean')
).round()

cluster_summary
```

Out[13]:

Cluster	Customers	Average_Age	Average_Income	Average_Spending
0	81	43.0	55.0	50.0
1	39	33.0	87.0	82.0
2	22	25.0	26.0	79.0
3	35	41.0	88.0	17.0
4	23	45.0	26.0	21.0

8. Silhouette Score

In [14]:

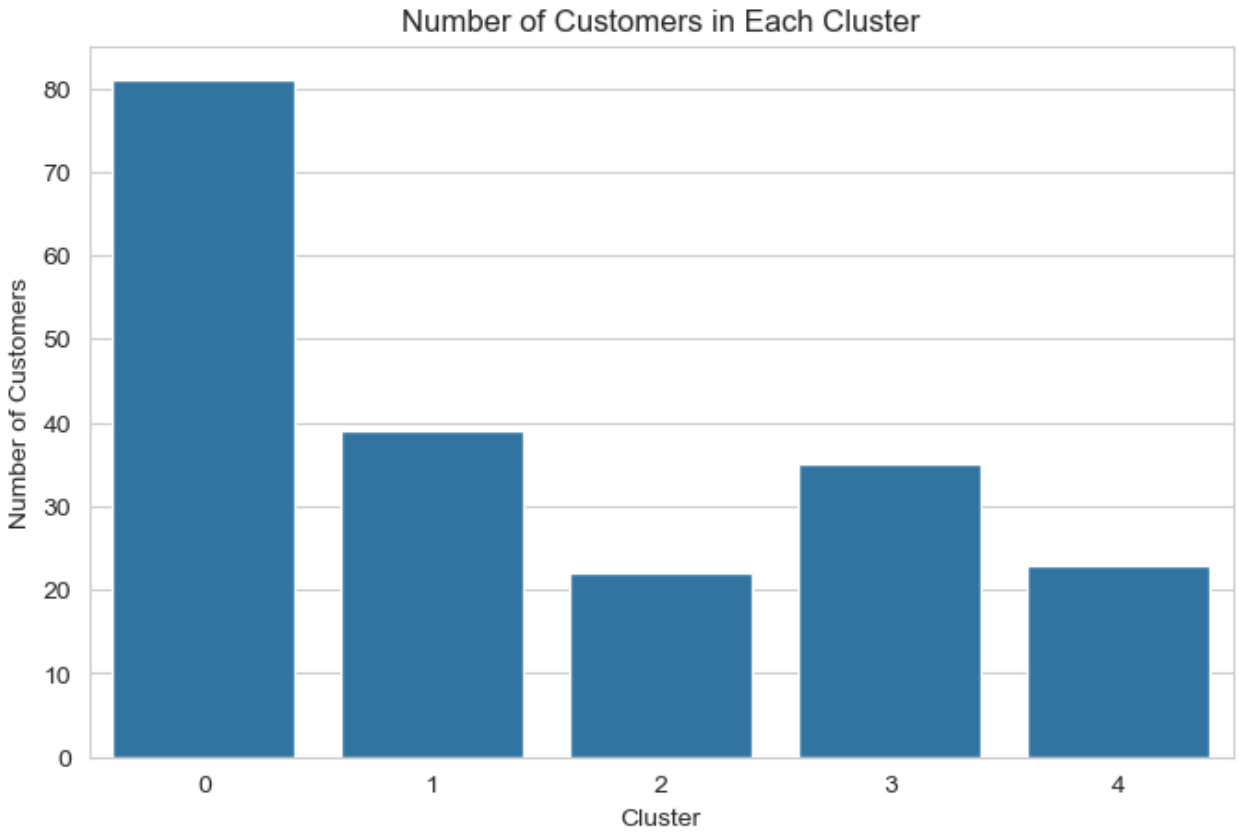
```
silhouette = silhouette_score(X_scaled, df['Cluster'])
print('Silhouette Score:', round(silhouette, 5))
```

```
Silhouette Score: 0.55466
```

9. Customers in Each Cluster

In [15]:

```
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='Cluster')
plt.title('Number of Customers in Each Cluster')
plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.show()
```



10. Conclusion

Business Insights from Customer Segmentation

The K-Means clustering algorithm identified five distinct customer segments based on their income and spending behavior. The model achieved a Silhouette Score of 0.55466, indicating reasonably good separation between the clusters. The results provide useful insights for developing targeted marketing strategies, improving customer engagement, and identifying opportunities to increase sales.

Cluster 0 – Moderate-Income, Moderate-Spending Customers: This is the largest cluster, containing 81 customers with an average age of 43 years, an average annual income of approximately $55k, and an average spending score of 50. These customers demonstrate moderate spending behavior and represent a substantial portion of the customer base. Businesses can target this segment with seasonal promotions, product recommendations, cross-selling opportunities, and loyalty programs to encourage additional purchases.

Cluster 1 – High-Income, High-Spending Customers: This cluster consists of 39 customers with an average age of 33 years, an average annual income of approximately $87k, and an average spending score of 82. These relatively younger customers have high incomes and high spending scores, making them a valuable segment for premium products, exclusive offers, personalized shopping experiences, and customer retention programs.

Cluster 2 – Low-Income, High-Spending Customers: This cluster contains 22 customers with an average age of 25 years, an average annual income of approximately $26k, and an average spending score of 79. These customers are relatively young and spend considerably despite their lower average incomes. Businesses can attract and retain them through affordable trendy products, flexible offers, student or youth-oriented promotions, and personalized recommendations while remaining mindful of their budget constraints.

Cluster 3 – High-Income, Low-Spending Customers: This cluster contains 35 customers with an average age of 41 years, an average annual income of approximately $88k, and an average spending score of 17. Although these customers earn relatively high incomes, their spending activity is low. Businesses could investigate the

these customers earn relatively high incomes, their spending activity is low. Businesses could investigate the reasons for their limited purchases and use targeted promotions, personalized recommendations, loyalty incentives, and premium product demonstrations to encourage greater engagement.

Cluster 4 – Low-Income, Low-Spending Customers: This cluster consists of 23 customers with an average age of 45 years, an average annual income of approximately $26k, and an average spending score of 21. These customers have relatively low incomes and low spending activity. Businesses can focus on affordable products, budget-friendly offers, discounts, and essential product recommendations to meet their needs without relying on expensive promotional strategies.

Overall Conclusion: The analysis demonstrates that customers have different demographic characteristics, income levels, and spending patterns and should not be treated as a single homogeneous group. The five segments enable businesses to distinguish between high-value customers, customers with potential for increased spending, and customers who may respond better to budget-friendly offerings. By tailoring marketing campaigns, pricing strategies, and product recommendations to each segment, businesses can allocate marketing resources more effectively, strengthen customer relationships, improve customer retention, and identify opportunities for revenue growth. The Silhouette Score of 0.55466 supports the usefulness of this clustering solution, although the final choice of five clusters should also consider the interpretability and practical business value of the resulting segments.

In [16]:

```
df.to_csv('ALL_customer_segments.csv', index=False)
print('Saved segmented dataset to ALL_customer_segments.csv')
```

```
Saved segmented dataset to ALL_customer_segments.csv
```

In [ ]:

```

```

ECommerce Sales Prediction:

A project by Om Satyawan Pathak

Contact: omsatyawanpathakgit@gmail.com

In [1]:

```
#Import the required modules:
import pandas as pd
import numpy as np
import joblib

from sklearn.model_selection import train_test_split, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import AdaBoostRegressor, RandomForestRegressor
from xgboost import XGBRegressor

from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

import warnings
warnings.filterwarnings("ignore")
```

In [2]:

```
#Load the dataset:
df = pd.read_csv("ecommerce_sales_dataset.csv")

df.head()
```

Out[2]:

	order_id	order_date	customer_id	product_category	region	quantity	unit_price	discount	payment_method	delivery_days
0	10001	1/1/2022	1102	Beauty	South	7	373.65	0.28	Wallet	10
1	10002	1/2/2022	1435	Clothing	South	7	47.74	0.09	Card	6
2	10003	1/3/2022	1860	Beauty	East	3	311.28	0.31	COD	6
3	10004	1/4/2022	1270	Electronics	West	5	524.47	0.02	Wallet	6
4	10005	1/5/2022	1106	Clothing	West	5	139.87	0.33	Wallet	4

In [3]:

```
#Inspect the dataset:
# Check the number of rows and columns
print("Dataset Shape:", df.shape)

# Check column names and data types
print("\n\nDataset information:")
df.info()

# Check statistical information
print("\n\nDescription of the dataset:")
df.describe()
```

```
Dataset Shape: (5000, 12)


Dataset information:
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   order_id          5000 non-null   int64  
 1   order_date        5000 non-null   object 
 2   customer_id       5000 non-null   int64  
 3   product_category  5000 non-null   object 
 4   region            5000 non-null   object 
 5   quantity          5000 non-null   int64  
 6   unit_price        5000 non-null   float64
 7   discount          5000 non-null   float64
 8   payment_method    5000 non-null   object 
 9   delivery_days     5000 non-null   int64  
 10  customer_rating   5000 non-null   float64
 11  revenue           5000 non-null   float64
dtypes: float64(4), int64(4), object(4)
memory usage: 468.9+ KB
```

Description of the dataset:

Out[3]:

	order_id	customer_id	quantity	unit_price	discount	delivery_days	customer_rating	revenue
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000
mean	12500.500000	1505.701200	4.044800	308.418774	0.179984	6.118800	2.973980	1021.955148
std	1443.520003	290.836902	2.020398	169.259369	0.101404	3.153264	1.157722	825.584219
min	10001.000000	1000.000000	1.000000	15.150000	0.000000	1.000000	1.000000	11.210000
25%	11250.750000	1253.000000	2.000000	161.895000	0.090000	3.000000	2.000000	354.527500
50%	12500.500000	1510.000000	4.000000	309.890000	0.180000	6.000000	3.000000	796.650000
75%	13750.250000	1761.000000	6.000000	455.557500	0.270000	9.000000	4.000000	1515.690000
max	15000.000000	1999.000000	7.000000	599.960000	0.350000	11.000000	5.000000	4119.330000

In [4]:

```
# Check missing values
print("No. of missing values:")
print(df.isnull().sum())

# Check duplicate records
print("\n\nDuplicate Rows:", df.duplicated().sum())

print("\n\nCheck unique values in categorical columns:")
print(df.select_dtypes(include="object").nunique())
```

```
No. of missing values:
order_id            0
order_date          0
customer_id         0
product_category    0
region              0
quantity            0
unit_price          0
discount            0
payment_method      0
delivery_days       0
customer_rating     0
revenue             0
dtype: int64


Duplicate Rows: 0
```

```
Check unique values in categorical columns:
order_date          5000
product_category       4
region                 4
payment_method         3
dtype: int64
```

In [5]:

```python
#Data Cleaning:


#Remove duplicate records
df = df.drop_duplicates()

# Convert order_date to datetime
df["order_date"] = pd.to_datetime(
    df["order_date"],
    errors="coerce"
)

# Remove records with missing target or invalid dates
df = df.dropna(subset=["revenue", "order_date"])

# Remove invalid numerical records
df = df[
    (df["quantity"] > 0) &
    (df["unit_price"] >= 0) &
    (df["discount"].between(0, 1)) &
    (df["revenue"] >= 0)
]

# Reset index
df = df.reset_index(drop=True)

print("Shape after cleaning:", df.shape)
print(df.isnull().sum())
```

```
Shape after cleaning: (5000, 12)
order_id            0
order_date          0
customer_id         0
product_category    0
region              0
quantity            0
unit_price          0
discount            0
payment_method      0
delivery_days       0
customer_rating     0
revenue             0
dtype: int64
```

In [6]:

```python
#Feature Engineering:


# Extract useful date features
df["order_year"] = df["order_date"].dt.year
df["order_month"] = df["order_date"].dt.month
df["order_day"] = df["order_date"].dt.day
df["order_dayofweek"] = df["order_date"].dt.dayofweek

# Separate features and target
X = df.drop(columns=[
    "revenue",
    "order_id",
    "customer_id",
    "order_date",
    "delivery_days",
    "customer_rating"
```

```
])

y = df["revenue"]

# Identify numerical and categorical columns
num_cols = X.select_dtypes(include=np.number).columns.tolist()
cat_cols = X.select_dtypes(include="object").columns.tolist()

print("Numerical Columns:", num_cols)
print("Categorical Columns:", cat_cols)
```

```
Numerical Columns: ['quantity', 'unit_price', 'discount', 'order_year', 'order_month', 'o
rder_day', 'order_dayofweek']
Categorical Columns: ['product_category', 'region', 'payment_method']
```

In [ ]:

```

```

In [ ]:

```

```

In [7]:

```
# Machine Learning part:
```

In [8]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (4000, 10)
X_test: (1000, 10)
y_train: (4000,)
y_test: (1000,)
```

In [9]:

```
num_cols = X_train.select_dtypes(include=np.number).columns
cat_cols = X_train.select_dtypes(include="object").columns

scaler = StandardScaler()

X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])

# One-hot encode categorical columns
X_train = pd.get_dummies(X_train, columns=cat_cols)
X_test = pd.get_dummies(X_test, columns=cat_cols)

# Ensure both datasets have the same columns
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)

print("Missing values in X_train:", X_train.isnull().sum().sum())
print("Missing values in X_test:", X_test.isnull().sum().sum())
print("Missing values in y_train:", y_train.isnull().sum())
print("Missing values in y_test:", y_test.isnull().sum())
```

```
Training data shape: (4000, 18)
```

```
Testing data shape: (1000, 18)
Missing values in X_train: 0
Missing values in X_test: 0
Missing values in y_train: 0
Missing values in y_test: 0
```

In [10]:

```
models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree Regressor": DecisionTreeRegressor(random_state=42),
    "Random Forest Regressor": RandomForestRegressor(
        random_state=42,
        n_estimators=100
    ),
    "AdaBoost Regressor": AdaBoostRegressor(random_state=42),
    "XGBoost Regressor": XGBRegressor(
        random_state=42,
        n_estimators=100
    )
}
```

In [11]:

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.metrics import mean_absolute_error, root_mean_squared_error, r2_score


kf = KFold(n_splits=5, shuffle=True, random_state=42)

results = {}

for name, model in models.items():

    r2_scores = cross_val_score(
        model, X_train, y_train,
        cv=kf, scoring="r2"
    )

    mae_scores = -cross_val_score(
        model, X_train, y_train,
        cv=kf, scoring="neg_mean_absolute_error"
    )

    rmse_scores = -cross_val_score(
        model, X_train, y_train,
        cv=kf, scoring="neg_root_mean_squared_error"
    )

    results[name] = {
        "R2 Score": r2_scores.mean(),
        "MAE": mae_scores.mean(),
        "RMSE": rmse_scores.mean()
    }

    print(name)
    print("Mean R2 Score:", r2_scores.mean())
    print("Mean MAE:", mae_scores.mean())
    print("Mean RMSE:", rmse_scores.mean())
    print("-" * 40)
```

```
Linear Regression
Mean R2 Score: 0.8672811426142172
Mean MAE: 221.4577159180697
Mean RMSE: 300.8223692814246
----------------------------------------
Decision Tree Regressor
Mean R2 Score: 0.9942467183084036
Mean MAE: 42.27419249999999
Mean RMSE: 62.590424512292635
----------------------------------------
```

```
Random Forest Regressor
Mean R2 Score: 0.9981543806546498
Mean MAE: 23.352572375000005
Mean RMSE: 35.296446780513676
----------------------------------------
AdaBoost Regressor
Mean R2 Score: 0.9479893661115085
Mean MAE: 159.0596835010242
Mean RMSE: 188.23228677799133
----------------------------------------
XGBoost Regressor
Mean R2 Score: 0.9984690818171178
Mean MAE: 22.274192639484404
Mean RMSE: 32.27866915062235
----------------------------------------
```

In [12]:

```
results_df = pd.DataFrame(results).T
results_df = results_df.sort_values(by="R2 Score", ascending=False)

print(results_df)

best_model_name = results_df.index[0]
best_model = models[best_model_name]

print("Best Model:", best_model_name)

#Train the best model:
best_model.fit(X_train, y_train)
```

```
                           R2 Score          MAE         RMSE
XGBoost Regressor          0.998469    22.274193    32.278669
Random Forest Regressor    0.998154    23.352572    35.296447
Decision Tree Regressor    0.994247    42.274192    62.590425
AdaBoost Regressor         0.947989   159.059684   188.232287
Linear Regression          0.867281   221.457716   300.822369
Best Model: XGBoost Regressor
```

Out[12]:

▾ XGBRegressor

i ?

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=None,
             enable_categorical=False, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=None, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
             max_leaves=None, min_child_weight=None, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=10
```

In [13]:

```
#Evaluate the best model on test-data:
y_pred = best_model.predict(X_test)

print("Best Model:", best_model_name)
print("R2 Score:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", root_mean_squared_error(y_test, y_pred))
```

```
Best Model: XGBoost Regressor
R2 Score: 0.9984260630041332
MAE: 22.586621254634853
RMSE: 32.56475327993813
```

In [14]:

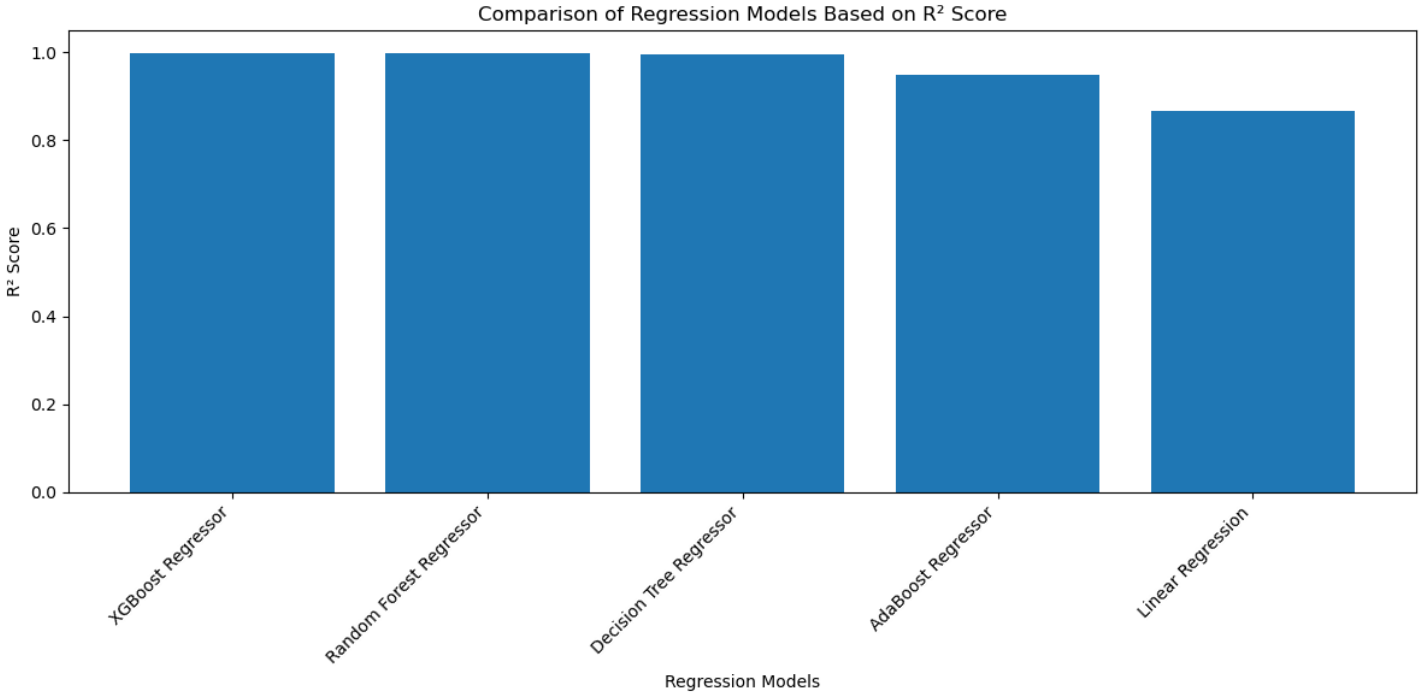
```
#Model Comparison based on R2 Score:
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

plt.bar(results_df.index, results_df["R2 Score"])

plt.title("Comparison of Regression Models Based on R² Score")
plt.xlabel("Regression Models")
plt.ylabel("R² Score")

plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



In [15]:

```
#Make the best model to do predictions for new data:


new_data = pd.DataFrame({
    "product_category": ["Beauty"],
    "region": ["East"],
    "quantity": [3],
    "unit_price": [311.28],
    "discount": [0.31],
    "payment_method": ["COD"],
    "order_year": [2022],
    "order_month": [3],
    "order_day": [1],
    "order_dayofweek": [2]
})

# Scale numerical columns:
new_data[num_cols] = scaler.transform(new_data[num_cols])

# One-hot encode categorical columns:
new_data = pd.get_dummies(new_data, columns=cat_cols)

# Match the training columns:
new_data = new_data.reindex(columns=X_train.columns, fill_value=0)

# Predict revenue:
predicted_revenue = best_model.predict(new_data)

print("Predicted Revenue:", predicted_revenue[0])
```

```
Predicted Revenue: 643.4581
```

In [ ]:

```
```

In [ ]:

```
```

In [16]:

```
#Make the best model to do predictions for another new data:


new_data = pd.DataFrame({
    "product_category": ["Clothing"],
    "region": ["West"],
    "quantity": [5],
    "unit_price": [139.87],
    "discount": [0.33],
    "payment_method": ["Wallet"],
    "order_year": [2022],
    "order_month": [5],
    "order_day": [1],
    "order_dayofweek": [7]
})

# Scale numerical columns:
new_data[num_cols] = scaler.transform(new_data[num_cols])

# One-hot encode categorical columns:
new_data = pd.get_dummies(new_data, columns=cat_cols)

# Match the training columns:
new_data = new_data.reindex(columns=X_train.columns, fill_value=0)

# Predict revenue:
predicted_revenue = best_model.predict(new_data)

print("Predicted Revenue:", predicted_revenue[0])
```

```
Predicted Revenue: 488.5463
```

In [ ]:

```
```

In [17]:

```
#Save the best-trained model and preprocessing objects:

joblib.dump(best_model, "best_revenue_prediction_model.pkl")

joblib.dump({
    "num_cols": list(num_cols),
    "cat_cols": list(cat_cols),
    "scaler": scaler,
    "feature_columns": list(X_train.columns)
}, "data_preprocessing.pkl")

print("Model and preprocessing objects saved successfully.")
```

```
Model and preprocessing objects saved successfully.
```

In [ ]:

```
```

In [ ]:

```
```

IMDB Movie Reviews Sentiment Analysis:

A Project by Om Satyawan Pathak

In [1]:

```
#Import the required modules:

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import wordcloud
import nltk

#Ignore all incoming warnings (if any):
import warnings
warnings.filterwarnings("ignore")
```

In [2]:

```
#Download the stop-words:
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Lenovo\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[2]:

```
True
```

In [3]:

```
from nltk.corpus import stopwords
```

In [4]:

```
#Import the dataset:
data = pd.read_csv("Movie_Review.csv")
```

In [5]:

```
data
```

Out[5]:

	text	sentiment
0	My daughter liked it but I was aghast, that a ...	neg
1	I... No words. No words can describe this. I w...	neg
2	this film is basically a poor take on the old ...	neg
3	This is a terrible movie, and I'm not even sur...	neg
4	First of all this movie is a piece of reality ...	pos
...	...	...
995	"Campfire Tales" is basically made up of three...	pos
996	I just saw this film at the phoenix film festi...	pos
997	It's out of question that the real Anna Anders...	pos
998	Michelle Pfeiffer stars as a mob widow who see...	pos
999	This film tells the stories of several couples...	pos

	text	sentiment

1000 rows × 2 columns

In [6]:

```
#Check the distribution in the target variable:
data.sentiment.value_counts()
```

Out[6]:

```
sentiment
neg    500
pos    500
Name: count, dtype: int64
```

In [7]:

```
#Check for any missing values:
data.isnull().sum()
```

Out[7]:

```
text         0
sentiment    0
dtype: int64
```

In [8]:

```
#Check for any duplicate values:
data.duplicated().sum()
```

Out[8]:

```
np.int64(1)
```

In [9]:

```
#Drop all duplicate values:
data.drop_duplicates(inplace=True)
```

In [10]:

```
#Check again for any duplicate values:
data.duplicated().sum()
```

Out[10]:

```
np.int64(0)
```

In [ ]:

```

```

In [11]:

```
#Preprocess the text attribute:
def clean_review(review):
    str = ' '.join(word for word in review.split() if word.lower() not in stopwords.word
s('english'))
    return str
data['text'] = data['text'].apply(clean_review)
```

In [12]:

```
#Check for the most frequent words:
from wordcloud import WordCloud

reviews_neg = ' '.join(word for word in data['text'][data['sentiment'] == 'neg'].astype(
str))
wordcloud_neg = WordCloud(height=600, width=1000, max_font_size=100).generate(reviews_neg
)
```

```
reviews_pos = ' '.join(word for word in data['text'][data['sentiment'] == 'pos'].astype(
str))
wordcloud_pos = WordCloud(height=600, width=1000, max_font_size=100).generate(reviews_pos
)


plt.figure(figsize=(15, 6))

plt.subplot(1, 2, 1)
plt.imshow(wordcloud_neg, interpolation='bilinear')
plt.title('All Negative Sentiments')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(wordcloud_pos, interpolation='bilinear')
plt.title('All Positive Sentiments')
plt.axis('off')

plt.tight_layout()
plt.show()
```



In [13]:

```
#Convert the reviews to numerical format:

from sklearn.feature_extraction.text import TfidfVectorizer
cv = TfidfVectorizer(max_features=2500,stop_words="english")
reviews = cv.fit_transform(data['text']).toarray()
```

In [14]:

```
#Convert the sentiments to numerical format:
data['sentiment'] = data['sentiment'].replace(['pos','neg'],[1,0])
```

In [15]:

```
data['sentiment'].value_counts()
```

Out[15]:

```
sentiment
1    500
0    499
Name: count, dtype: int64
```

In [16]:

```
from sklearn.model_selection import train_test_split, cross_validate, KFold

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier , ExtraTreesClassifier
from xgboost import XGBClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.metrics import classification_report , confusion_matrix
```

In [17]:

```
X = reviews
y = data["sentiment"]

X_train, X_test , y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=4
2)

models = {
    "Logistic Regression":LogisticRegression(max_iter=1000,random_state=42),
    "Decision-Tree Classifier":DecisionTreeClassifier(max_depth=3,random_state=42),
    "AdaBoost Classifier":AdaBoostClassifier(estimator=DecisionTreeClassifier(max_depth=6
),
        n_estimators=200,
        learning_rate=0.8,
        random_state=42),
    "Extra-Trees Classifier":ExtraTreesClassifier(
        n_estimators=130, max_depth=4, n_jobs=-1, random_state=42
    ),
    "XGBoost Classifier":XGBClassifier(
        n_estimators=300,
        learning_rate=0.1,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=42,
        n_jobs=-1,
        tree_method="hist",
        eval_metric="logloss"
    ),
    "KNN Classifier":KNeighborsClassifier(n_neighbors=8, n_jobs=-1)
}
```

In [18]:

```
kf = KFold(n_splits=5,shuffle=True,random_state=42)
scoring = ["accuracy","f1_weighted"]

results=[]

for name,model in models.items():
    print(f"{name} is running .......")
    cv_results = cross_validate(model,X_train,y_train,cv=kf,scoring=scoring,n_jobs=-1)

    results.append({
        "Model":name,
        "CV Accuracy":round(np.mean(cv_results["test_accuracy"]),4),
        "CV F1 Score":round(np.mean(cv_results["test_f1_weighted"]),4)
    })

results_df = pd.DataFrame(results).sort_values(by="CV Accuracy",ascending=False).reset_i
ndex(drop=True)



display(results_df)
```

```
Logistic Regression is running .......
Decision-Tree Classifier is running .......
AdaBoost Classifier is running .......
Extra-Trees Classifier is running .......
XGBoost Classifier is running .......
KNN Classifier is running .......
```

	Model	CV Accuracy	CV F1 Score
0	**Logistic Regression**	**0.7922**	**0.7925**
1	**AdaBoost Classifier**	**0.7434**	**0.7436**

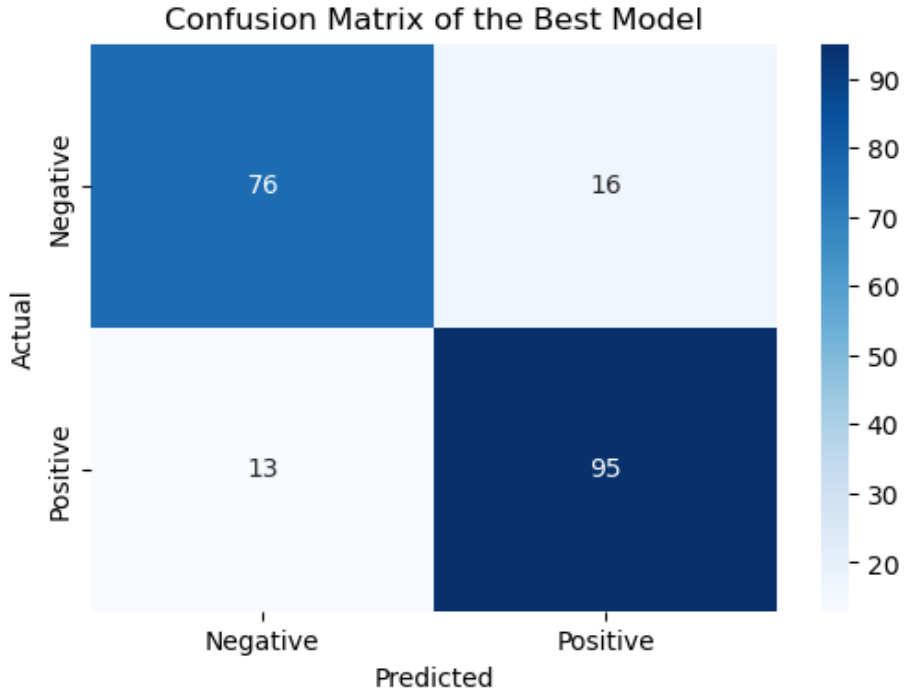
	Model	CV Accuracy	CV F1 Score
2	XGBoost Classifier	0.7421	0.7425
3	Extra-Trees Classifier	0.7194	0.7051
4	KNN Classifier	0.6821	0.6732
5	Decision-Tree Classifier	0.6196	0.5857

In [19]:

```
#Evaluate the best model (i.e. Logistic Regression) on the test set:
best_model = models["Logistic Regression"]
best_model.fit(X_train,y_train)

y_pred = best_model.predict(X_test)

cm = confusion_matrix(y_test,y_pred)
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix of the Best Model")
plt.show()
print("Classification Report of the best model:")
print(classification_report(y_test, y_pred, target_names=["Positive", "Negative"]))
```




```
Classification Report of the best model:
              precision    recall  f1-score   support

    Positive       0.85      0.83      0.84        92
    Negative       0.86      0.88      0.87       108

    accuracy                           0.85       200
   macro avg       0.85      0.85      0.85       200
weighted avg       0.85      0.85      0.85       200
```

In [20]:

```
#Export this model and the vectorizer:
import pickle as pk
```

```
pk.dump(best_model,open("movie_review_sentimentPredictor.pkl","wb"))
pk.dump(cv,open("tfidf_vectorizer.pkl","wb"))
```

In [21]:

```
with open("movie_review_sentimentPredictor.pkl", "rb") as file:
    model = pk.load(file)

with open("tfidf_vectorizer.pkl", "rb") as file:
    cv = pk.load(file)

review = input("Enter a movie review: ")

review_vector = cv.transform([review]).toarray()
prediction = model.predict(review_vector)


if prediction[0]==1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Negative Review
```

In [22]:

```
review = input("Enter a movie review: ")

review_vector = cv.transform([review]).toarray()
prediction = model.predict(review_vector)


if prediction[0]==1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Negative Review
```

In [23]:

```
review = input("Enter a movie review: ")

review_vector = cv.transform([review]).toarray()
prediction = model.predict(review_vector)


if prediction[0]==1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Positive Review
```

In [24]:

```
review = input("Enter a movie review: ")

review_vector = cv.transform([review]).toarray()
prediction = model.predict(review_vector)


if prediction[0]==1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Positive Review
```

In [25]:

```
review = input("Enter a movie review: ")
```

```
review_vector = cv.transform([review]).toarray()
prediction = model.predict(review_vector)


if prediction[0]==1:
    print("Positive Review")
else:
    print("Negative Review")
```

```
Negative Review
```

In [ ]:

```
```

ABOUT ME:



Name: Om Satyawan Pathak
Email: omsatyawanpathakgit@gmail.com
Github: https://github.com/omsatyawanpathakgit
Linkedin: www.linkedin.com/in/om-satyawan-pathak-029b02368